C++

FREEMIND

Date

Page

- C++ Basics

⇒ Skelton of C++

cout => used to print

↓

Console Out

```
#include <iostream>
        Library
int main ()
{
    cout << "Hello world";
    return 0;
}
```

↑ insertion Operator

iostream
input/Output Stream

first.cpp

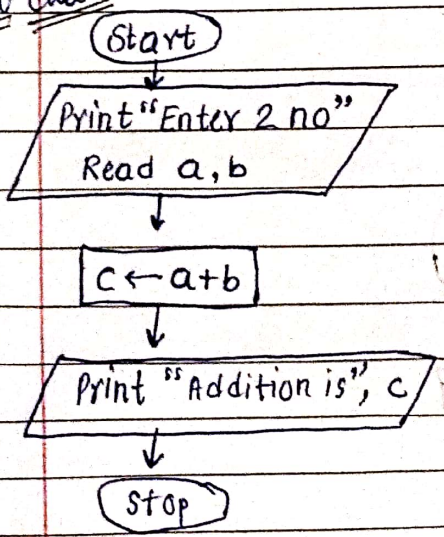
```
#include <iostream>
using namespace std;
int main ()
{
   Cout << "Hello world";
   return 0;
}
```

or

```
#include <iostream>
int main()
{
  Std :: cout << "Hello world";
  return 0;
}
```

Sum of 2 Numbers

Flow Chart




```
#include <iostream>
int main() {
int a, b, c;
Std :: cout << "Enter 2 number";
Std :: cin >> a >> b;
c = a+b;
std :: cout << "The addition is" << c;
return 0;
}
```

Scanned with OKEN Scanner

Que WAP To Take user name

```
#include <string>
#include <iostream>
int main()
{
    string name;
    cout<<"May I know Your name";
    getline(cin,name);
    cout<<"Welcome Mr "<<name;
    return 0;
}
```

cin >> name;
↓
Read till WhiteSpace

getline (cin, variable_name);
↓
Read whole line

Data type

data
- Numeric
  - 12
  - 25
  - 15.35
  - Integer
  - floating
- Character
  - Alphabetic
  - 125-A
  - Mehul, Delhi, Jaipur
  - String

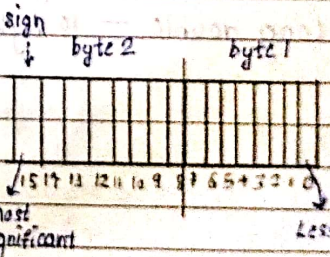
1 byte → 8 bit

0 – 255

Datatypes
- Primitive
  - Integral
    - int
    - char
  - bool
  - floating Point
    - float
    - double
- userdefined
  - enum
  - Structure
  - Union
  - class
- Derived
  - Array
  - Pointer
  - Reference

data types	Size	Range
int	2 or 4	-32768 to 32767
float	4	-3.4×10^{-38} to 3.4×10^{38}
double	8	-1.7×10^{-308} to 1.7×10^{308}
char	1	-128 to 127
bool	undefined	true / false

int



1 -ve $2^{15} = 32768$

0 +ve 0 - 32767

Scanned with OKEN Scanner

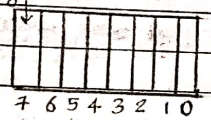
FREEMIND

Date

Page

char

Sign



$2^7 = 128$

0 - 127

-128 to 127

ASCII

A - 65 a - 97 0 - 48

B - 66 b - 98 1 - 49

Z - 90 z - 122 9 - 57

Modifier

- unsigned
- long

unsigned int
0 - 65535

unsigned char
0 - 255

Long int — 4 bytes
— 8 bytes

Long double — 10 bytes

FREEMIND

Date

Page

Variable

```
int rollno = 10;
char group = 'A';
float price = 12.75f;
```

If You dont write this then By default it will consider as double

Valid and invalid name

int xc1; ✓ int 1x; X

int rollno; ✓ int roll no ; X

int roll_no; ✓

int rollNo; ✓

int RollNo; ✓

Operators & Expressions

Arithmetic — +, -, *, /, %

Relational — <, <=, >, >=, ==

Logical — &&, ||, !

Bitwise — &, |, ~, ^

Increament — ++, --

decreament

Assignment — =

Can't Perfor Mod on float

```
#include <cmath>
#include<math.h>
```

sqrt ( );

pow (x, y)

x^y → 2^3

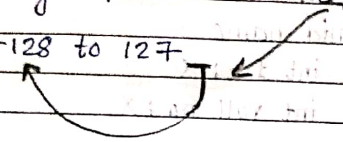
Overflow

```
char x = 127
++x;
```

If we increase a then it will give this

Range of char

-128 to 127



Bitwise operator

AND (&)

Performs a bitwise AND operations

x = 5 , y = 3 , z

```
 0101
 0011
 0001
```

z = x & y

= 1

OR (|)

Performs a bitwise OR operation

x = 5 , y = 3 , z = x | y

```
 0101
 0011
 0111
```

z = 7

XOR (^)

Performs a bitwise XOR operations (Exclusive OR)

x = 5 , y = 3 , z = x ^ y

```
 0101
 0011
 0110
```

z = 6

NOT (~)

Performs a bitwise NOT (complement) operation

~5

~0101

= 1010 (in 2's complement), result is -6

Left Shift (<<)

Shift bits to left

a = 5

a<<1

00000101 << 1 = 00001010

a<<2

00000101 << 2 = 00010100

Right Shift (>>)

Shift bits to Right

a = 20

a >> 1

00010100 >> 1 = 00001010

a = 10

a >> 2

00010100 >> 1 = 00000101

a = 5

Assignment

a &= b

a |= b

a ^= b

a >>= b

a <<= b

Enum and Typedef

1. Enum (enum) (Short for enumeration)
is a user defined data type in C/C++ that consists of integral constants, providing a way to define named integer constants for better code readability.

Syntax

```
enum EnumName { constant1, constant2, constant3, .... };
```

Example

```
enum Days { Monday, Tuesday, Wednesday, Thursday, Friday };
```
(0 1 2 3 4)

Can also assign Specific Values

```
enum Colours { Red = 1, Green = 5, Blue = 10 };
```

2. Typedef (typedef):
typedef is keyword in C/C++ that allows you to create new names (aliases) for existing data types.

Syntax:

```
typedef existingType newTypeName;
```

Example

```
typedef unsigned int uint
uint age = 25;
```

```
#include<iostream>
using namespace std;
typedef int marks;

int main()
{
    marks m1, m2;
    m1 = 50;
    m2 = 90;

}
```

End

Enum	Typedef
Defines set of named constant	Creates a new name (alias) for an existing type
Values are automatically assigned integers	Does not assign values, just creates aliases
Used for grouping constants	Used for simplifying types names.

Conditional Statement

Same as C language

if

```
if (condition) {
    // code to be Executed
}
```

if-else

```
if Condition{
    // code if cond^n is True
}else {
    // code if cond^n is false
}
```

if-else if

```
if (Condition1) {
    // if cond^n 1 is True
}else if (condition2) {
    // if 2nd is True
}else {
    // if both or none is True
}
```

Switch

```
Switch (Variable){
case Value1:
    // for case Value 1
    break;
Case Value 2:
    // for case Value 2
    break;
default
    // for if no
```

Ternary (?:)

condition ? expression1 : expression 2;

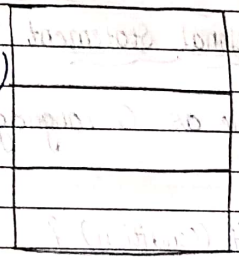
Short Circuit

if (a>b && a>c)
F X
If 1st is False 2nd will not be checked

if (a>b || a>c)
T X
If 1st is True 2nd will not be Checked

Dynamic declaration

In C++ Variable declared inside block (e.g within {}) have local Scope and are automatically destroyed Once the block ends. These are automatic Variables Stored in the Stack Memory not the Heap



Example of Block Scoped Variables

```
Void my function(){
  if(true){
    int x=10;
    cout<<x<<endl; //output: 10
  }
  // x is deleted here (end of block), no longer accessible
  // cout<<x; // This will cause an Error
}
```

Pointers

A pointer in C++ is a Variable that Stores the memory address of another Variables.

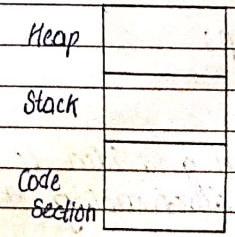
1. Declaration
   int *ptr; null Pointer
2. Initialization
   int num = 10;
   int *ptr = #
   int *ptr = nullptr;
   Pointer that does not point to any Valid memory location.
3. Dereferencing
   cout<<*ptr;

Example

```
int a=10;
int *ptr=&a;

cout<<a<<endl;      // 10
cout<<&a<<endl;     // 0x7ff...e8
cout<<ptr<<endl;    // 0x7ff...e8
cout<<&p<<endl;     // 0x7ff...e0
cout<<*p<<endl;     // 10
```

Why pointers ? (Indirectly)



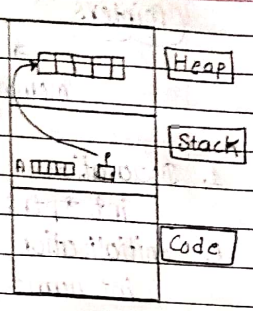
1. for Accesing Heap memory
2. for Accesing file in disk
3. for Accessing Network connection

4 for Keyboard, monitor, Printer etc

Scanned with OKEN Scanner

How to allocate in Heap

Heap



Stack → `int A[5] = {1,2,3,4,5};`

`int *p;`

Heap →

`p = new int[5];`

OR

`int *p = new int[5];`

In C++, Heap memory Refers to the area of memory used for dynamic ~~Holding~~ memory allocation. Unlike Stack memory, which is managed automatically (Variables are created and destroyed as they go in and out of Scope), Heap memory is manually managed by the Programmer using the new and delete operators.

Allocating Memory in Heap

Example

```
int *arr = new int[5]; //Allocates an array of 5 integers on the Heap
arr[0] = 10; //Assign the Values to the arrays
```

Deallocating Memory in Heap

```
delete[] arr;      delete p;
```

After deleting a pointer its often good idea to set it to nullptr to avoid accidental dereferencing of dangling pointer

Dynamic memory in Heap

What is dynamic memory in Heap?

⇒ Dynamic memory is created in Heap using pointers.

We can use Dynamic memory for creating Arrays, linked list, Trees, Map etc (These are called as Data structures)

Heap memory can be Accessed from anywhere in the program, if its address is available.

Heap memory can be used for storing data of Entire application.

Application of Dynamic memory?

1. Data Structures
2. Dynamic Arrays
   (If the size of array is not known at compile time dynamic memory allocation allows creating arrays of Varying Sizes during Runtime.)
3. Handling User input
   (when the size of user input (like Strings or numbers) is Unknown)
4. Flexible memory ~~allows~~ Management in Big Programs
   (Dynamic programs allows large programs (like games, Simulations or database Systems) to manage memory more efficiently by allocating and deallocating memory as needed.
5. File Handling
6. OOP
7. Game development

Scanned with OKEN Scanner

Memory leak:

A memory leak occurs when a program allocates memory on the Heap but forgets to free it after use. This cause memory remain allocated, even though it is no longer needed, leading to wasted memory

e.g.
```
{
int *ptr = new int[10];
return 0;          // memory leaks occur because the pointer 'ptr' is not freed
}
```

How to fix

use

```
delete ptr;
```

Dangling pointer:

A dangling pointer occurs when a pointer continues to a point to a memory location that has already been freed or deleted. using such a pointer can lead to unpridictable behaviour and crashes.

After → `delete ptr;`

use this too → `ptr = nullptr;`

Stack vs Heap memory

Stack:

- All the variables we declared in a function are created in stack.
- All variables are accessible only within that function
- Compare can finalise the amount of memory required for all variables
- When function is called, memory is created in stack
- When a function terminates memory deleted from stack

Heap

- Pointer is used for allocating memory at runtime
- new operator is used for allocating memory in Heap.
- Heap memory should be allocated when required and deallocated when not in use
- Heap memory can be accessed by entire program if pointer is available.

NULL vs nullptr

NULL

- It is a constant whose value is 0
- NULL means, pointer is not pointing on any valid location
- In place of NULL, 0 can be used
- Using 0 in place of NULL may create confusion for Programmer

nullptr

- It is a keyword in C++
- nullptr means, pointer in not pointing any valid location (same as NULL)

Scanned with OKEN Scanner

- NULL: Nullptr doesn't means 0
- 0 cannot be used in its place.

1. Stack (Arrays)
   - Fixed Size: Once an array is defined, its size cannot be changed.
   - Example: int arr[5]; You cannot change the size of array after this declaration
2. Heap
   - Resizable Size: You can allocate memory dynamically and resize it during runtime.
   - Example:

```
int *arr = new int[5];
delete[] arr;
arr = new int[new_size];
```

Problems using Pointer

1. Uninitialized Pointer
2. Memory leak
3. Dangling pointer

Reference — alias means giving an alternate name to an existing variable.

A Reference is an alias for another variables. Once a Refrence is initialized to a variables. It acts as an alternative name for that variable allowing direct access to the original value variable

```
main()
{
    int x = 10;
    int &y = x;

    int a; x++; → 11
           y++; → 12
    a = x; cout<<x; → 12
           cout<<y; → 12

    int a;
```

x/y ← Alias

10
200/201

Reference doesn't Consume any memory

&y = z ✗

data of x

r Value → a = x;

l Value → x = 25;

Address of x

For Each loop

In C++ the range-based for loop (often Refered to as a "for Each" loop)

Syntax

```
for(element_declaration : range) {
    //
}
```

example

```
int num[] = {1,2,3,4,5};
for(int x : num) {
    cout<<x<<" ";
}
```

Scanned with OKEN Scanner

String

Two Method

1. Using char array
2. class String

Using char array

```
char x = 'A';
char S[6] = "Hello";
char S[] = "Hello";
char S[] = {'H','e','l','l','o','\0'};
char S[] = {65,66,67,68,'\0'};
char *S = "Hello";
```

Reading and printing String

```
char s[20];
cout<<"Enter your name";
cin>>s;
char s[50];
cout<<"Enter your name";
cin.get(s,50);
cin.getline(s,50);
cout<<s;
```

Cin.ignore

- When we enter any input from keyboard, it is transfered in input buffer
- program reads any data from input buffer
- After entering value from keyboard, we hit enter
- Program will read the value and ignore enter key from buffer.
- If program doesn't ignore it can it may not read next input
- cin.ignore() is used for forcing the program to ignore it
- Usually program don't read a string twice because of enter key
- Use cin.ignore() for reading a string

getline(cin,str) is used for reading a String object. It will not work for char array

cin.getline(str,100) will not work for String class object

#include<cstring> this library contains a language function
#include<string> this contains user class for string

String

Two Method
1. String class string
2. class string

Using char array

```
char x = 'a';
char s[10] = "Rahul";
char s[] = "Rahul";
char s[] = {'R','a','h','u','l','\0'};
char s[] = {82, 97, 104, 117, 108, 0};
char *s = "Rahul";
```

Reading and printing String

```
char s[20];
cout<<"Enter your name";
cin>>s;
```

only — first word — stops at whitespace

```
char s[50];
cout<<"Enter your name";
cin.get(s,50);
cin.getline(s,50);
```

This will take whole name till size given

↓ to go to next line for null character ('\0')

```
cout<<s;
```

↓ Print all upwards till reach '\0'

cin.ignore

- When we scan any input from keyboard, it is stored in input buffer.
- program first scan data from input buffer.
- after entering value from keyboard, we hit enter.
- Program will read the value and ignore enter key from buffer.
- If program doesn't ignore it, it may get read next input.
- cin.ignore() is used for forcing the program to ignore it.
- Usually programs don't read a string value because of enter key.
- Use cin.ignore() for reading a string.

getline(cin,str) is used for reading a String object. It will not work for char array

cin.getline(str,100) will not work for String class object

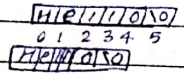
#include<cstring> this library contains c language function
#include<string> this contains c++ class for String

String

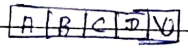
Two Method
1. Using char array
2. class string

Using char Array

```
char x = 'A';
char S[10] = "Hello";
char S[] = "Hello";
```




```
char S[] = {'H','e','l','l','o','\0'}
char S[] = {65, 66, 67, 68, '\0'}
char *S = "Hello";
```



Reading and printing String

```
char s[20];
cout<<"Enter Your name";
cin>>s;
```

Mehul Suthar
only — stops after space

```
char s[50];
cout<<"Enter Your name";
cin.get(s,50);
cin.getline(s,50);
cout<<s;
```

This will take whole name till size given

49 and one place for null character ('\0')

Print all alphabets till reach @ '\0'

Cin.ignore

- When we enter any input from keyboard, it is translated input buffer
- program Reads the data from input buffer
- After Entering Value from keyboard, we hit enter
- Program will Read the value and ignore enter key from buffer.
- If program doesn't ignore it the it may not Read next input
- cin.ignore() is used for forcing the ~~not~~ programs to ignore it
- Usually programs don't Read a String Value because of Enter Key
- Use cin.ignore() b4 reading a string

getline(cin,str) is used for reading a String object. It will not work for char array

cin.getline(str,100) will not work for String class object

#include<cstring> this library Contains c language function
#include<string> this Contains C++ class for String

Functions in String.h/cstring

* for finding length:

strlen(str2) str2: H e l l o \0 (1 2 3 4 5 6)

* for concanitation of string

strcat(destination, source);

and

strcat(destination, source, length);

How many character you want to concatinat

* for copying string

strcpy(destination, source);

and

strncpy(destination, source, length);

dest. → [ ]

Source → H e l l o \0

* for finding substring from main string

strstr(main, sub); → Return gramming (If sub was in main then it will Return)

main → P r o g r a m m i n g \0

Sub → g r a m \0

mming

* for finding occurence

strchr(main, char); Right side strrchr(main, char);

main: P r o g r a m m i n g \0

strchr(main, m) → Return → mming

strrchr(main, m) → Return → ming

* for comparing string

strcmp(str1, str2);

returns -ve, 0 or +ve

0: Str1 H e l l o \0 / Str2 H e l l o \0

-ve: Str1 h e l l o \0 / Str2 H e l l o \0

Str1 m i n o r \0

Str2 e l d e r \0

+ve

* strcmp(str1, str2)

If first letter in str1 come before 2nd string first letter the Return -ve value

If both Equal Return 0

If first letter in str1 come after 2nd string first letter the Return +ve value

or return difference of Ascii code

★ for converting string to number

Strtol(str1, NULL, 10/8/2) str1 | 2 | 3 | 7 | \0 |
str to long int

Strtof(str1, NULL) str | 5 | 3 | . | 7 | 6 | \0 |
str to float

```
Long int x = strtol(S1, NULL, 10)
float y = strtof(S2, NULL)
```

for decimal
for Octal → 8
for Binary → 2

★ for tokenizing Str

strtok(str1, "=?")

str1 | x | = | 10 | ; | y | = | 20 | ; | z | = | 35 | \0 |

```
int main()
{
  char S1[20] = "x=10;y=20;z=35";
  char *token = strtok(S1, "=;");
                       "?" or
  while(token != NULL)
  {
    cout<<token<<endl;
    token = strtok(NULL, "=;");
                         "?" or
  } return 0;
}
```

Output
x
10; y
20; z
35

Output
x=10; y=20; z=35

class string

#include<string>

string str; → This variable is called Object

Let string str = "Hello";

str | H | e | l | l | o | \0 | | | - - - - |
0 1 2 3 4 5 6 7 8 - - - - -
↑ length/size ↑ Capacity

Key features
- dynamically Resizable
- Supports various String operations
- Provide member functions for easy manipulation

Functions

String S2 = "World";
String S1 = "Hello";

1. S.length() } same — S1.length(); S2.length();
2. S.size()
3. S.capacity → It will Give the Capacity
4. S.resize(30) → Increase the capacity (can be set by us)
5. S.max_size() → maximum possible size
6. S.clear() → clear the Content of String
7. S.empty() → check String is Empty or not

Str = "programming";

8 S.append("Bye") → Added to the end of string
If Hello then HelloBye

9 S.insert(3,"KK") string str="Hello"; str.insert(3,"KK") HelKKlo
If str.insert(3,apple,3) Helapplelo

10 S.replace(3,5,"aa") S.replace(3,4,"KK") → Prokkaming

11 S.erase() → same like S.clear()

12 S.push_back('z') → insert single character at end
for eg Hello → Helloz

13 S.pop_back() → will remove last letter from string
Hello → Hell

14 S1.swap(S2) → will Exchange or Swap two string
for example S1 ↔ S2
S1 = "Hello" → S1 = "world"
S2 = "world" → S2 = "Hello"

15 S.copy(char des[]) → String "welcome";
char str[10];
S.copy(str, S.length());
Copy string to array
Wait, How to mention
How many characters
If I do like these
S.copy(str, 3) then I have to add null char there
str[3] = '\0';

only for string

16 S.find(str) or char
will find left to Right →
str

17 S.rfind(str)
will find from
right to left ←

S = "Hello World"
S.find("World"); → 6
S.find("llo"); → 2
S.find('o'); → 4
S.find(" "); → 5

18 S.find_first_of() → for specific character and
19 S.find_last_of() can also string but only show
the index of first letter only

It will show the index ~~which~~ B
which character come first among you inserted
for e.g.
S = "Hello World"
S.find_first_of("llo") → 2
coz l comes first

Or we can say it compare ("ow") → 4
all characters from word
coz o come first

and find_last_of() → will give index of last
character among string

* for Extracting portion of String

20 S.substr(start, number)
Str = "Programming"
0123 1234
str.substr(3); → gramming
str.substr(3,4); → gram

21 S.compare(str)

0 → if s equal to str
>0 ⇒ +ve value → if s is greater str
<0 -ve value → if s is smaller than str

Scanned with OKEN Scanner

Str = "programming";

9 S.append("Bye") → Added to the end of string
If Hello then HelloBye

8 S.insert(3,"xx") string str="Hello"; str.insert(3,"xx") Helxxlo
If str.insert(3,apple,2) Helapplelo

10 S.replace(3,5,"aa") S.replace(3,4,"xx") → Exercising

11 S.erase() → Same like S.clear()

12 S.push_back('z') → will insert single character at end
for eg Hello → Helloz

13 S.pop_back() → will remove last letter from string
Hello → Hell

14 S1.swap(S2) → will exchange or swap two string
for example S1 ⇄ S2
S1 = "Hello" → S1 = "world"
S2 = "world" → S2 = "Hello"

15 S.copy(char des[]) → String "Welcome";
char str[10];
S.copy(str, S.length());
↓
Here, Have to mention
How many characters
copy string to array

If I do like these

S.copy(str, 3) the I have to add null char there
Str[3] = '\0';

Only for String

16 S.find(str) or char
will find left to Right →
S = "Hello World"
S.find("World"); → 6
S.find("llo"); → 2
S.find('o'); → 4
S.find(" "); → 5

17 S.rfind(str)
will find from
right to left ←

18 S.find_first_of() → for specific character and
19 S.find_last_of() can also string But only shows
the index of first letter only

It will show the index which character come first among you inserted
for eg.
S = "Hello World"
S.find_first_of("llo") → 2
coz l comes first

Or we can say it compare ("ow") → 4
all characters from word
coz o come first

and find_last_of() → will give index of last
character among string

* for Extracting Portion of String

20 S.substr(start, number)
Str = "Programming"
0123 1234
str.substr(3); → gramming
str.substr(3,4); → gram

21 S.compare(str)
0 → If s equal to str
>0 → +ve Value → If S is greater str
<0 -ve Value → If s is smaller than str

Scanned with OKEN Scanner

22 str.at() → String str = "Holiday";
0 1 2 3 4
str.at(4); → d
or
str[4]; → d
str[] ↓
overloaded operator

Str1 = "Hello";
Str2 = "World";

S3 = Str1 + Str2;
and
also

Str1 = Str2
copy the entire string

Str1 = Str1 + "World

23 front() → will give the first letter of string
24 back() → will give the last letter of string

String iterator

String::iterator
begin()
end()

str | t | o | d | a | y | \0 | ----

String::iterator it;

for (it = str.begin(); it != str.end()
; it++) {
cout << *it;
}

String::reverse_iterator
rbegin()
rend()
→ same as this

functions

* functions are Blocks of code that perform specific tasks and Called multiple times to avoid redundancy.
They help in modular programming and Code Reusability.

We should avoid ~~using~~ user interaction
avoid cin and cout
This called bad function

Types of function

1. Built in functions – Predefined function
(e.g. sqrt(), pow(), abs()).
2. User-defined functions – functions defined by the user to Perform Specific tasks
3. Recursive functions – function that Call Themselves

function Syntax
```
returnType functionName (Parameters) {
    // function Body
    return value;
}
```

function overloading

function Overloading is a feature in C++ (and Some other languages) that allows multiple functions to have the Same name but diffrend parameters.
The Compiler decides (determines) Based on the number and types of arguments passed.

Scanned with OKEN Scanner

~~Function Ex~~

```
int add (int a, int b) {
    return a+b;
}
float add (float a, float b) {
    return a+b;
}
int add (int a, int b, int c) {
    return a+b+c;
}
```

Function Template

A function template in C++ allows You to Create a Single function definition that works ~~in~~ With diffrent data types.
It Enables genric programming, reducing code duplication.

Syntax

```
template <typename T>
T function_name (T Parameters) {
    // function Body
}
```

(class)

T acts as placeholder for any datatype

```
template <class T>
~~T max (T a, T b)~~
T max (T a, T b) {
    return a+b;
}
```

will take int or float or any other data type

Default Argument

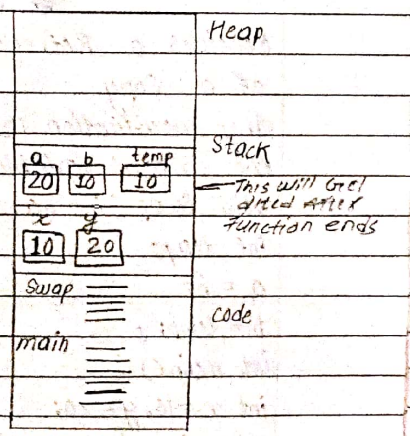
In C++ default Arguments allow You to Specify default Values for function parameters.
If function Call omits arguments, the default Values are used.

```
int add (int a=10, int b=20, int c=30) {
  return a+b+c;
}
int main () {
   add();        // Output 60
   add(5);       // Output 55
   add(5,15);    // Output 50
   add(5,15,25); // Output 45
   return 0;
}
```

Parameter Passing

1. Pass by Value:

```
void swap (int a, int b) {
int temp = a;
a = b;
b = temp;
}
int main () {
int x=10; y=20;
swap (x, y)
cout<<x<<" "<<y;
}
```

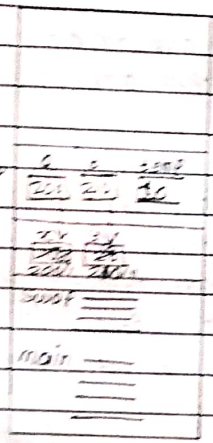


only Copy of actual Value is made Passed. Any change made to Parameter inside the function do not affect the Orignal Variable.

Pass by address

In pass by address, a function receives the memory address (pointer) of the actual argument, allowing direct modification of the original variable.

```
void Swap (int *a, int *b){
int temp;
temp = *a;
*a = *b
*b = temp;
}
int main () {
int x=10, y=20;
Swap(x,y);
cout << x << " " << y;
}
```



Pass by Refrence

In pass by Refrence, a function receives a Refrence to the original variable instead of a copy.
Any modification inside the function directly affects the original variable.

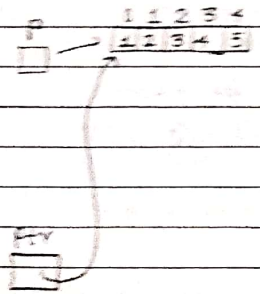
```
void Swap (int &a, int &b){
int temp;
a = b;
b = temp; }
int main() {
int x=10, y=20;
swap (x,y); <actual
cout << x << " " << y;
}
```

Return by address

In C++, returning by address means Returning a pointer to a variable instead of the actual value.

- This is useful when you want to return dynamically allocated memory a variable from a function.

```
int *fun(int size)
{
 int *P = new int[size];
 for(int i=0; i<size; i++){
      P[i] = i+1;
 } return P;
}
int main()
{
   int *ptr = fun(5);
 for (i=0; i<5; i++){
   cout << ptr[i] << endl;
   }
   delete[] ptr;
   ptr = nullptr;
   return 0;
}
```



Output:

0
5
10
15
20

Return by Reference

```
int & fun(int &x)
{
   return x;
}
int main()
{
   int a=10;
   fun(a)=25;
   cout<<x;
}
```

a/x
10/25

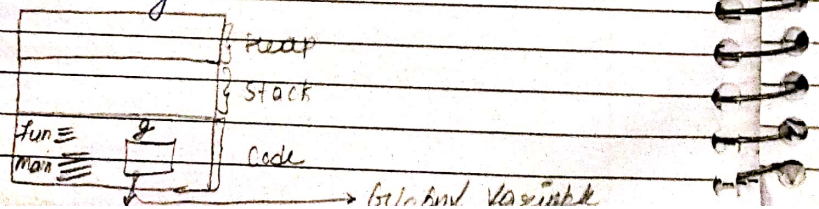
Global vs Local Variables

Global Variable

A Global Variable is declared outside all functions and is accessible throughout the program.
It is Stored in the data Segment and Retains its Value until the Program terminates

Local Variable

A Local Variable is declared inside a function and is accessible only within that function. ~~and~~
It is Stored in the Stack and Exists only while the function is Executing



~~Scope~~ Scoping Rule

Static Variable

```
void fun()
{
   static int v=0;
   int a=5;
   v++;
   cout<<a<<"  "<<v;
}
int main()
{
   fun() → 5 1
   fun() → 5 2
   fun() → 5 3
```

A Static Variable Retain it Value between function Calls and is Stored in the data Segment instead of the Stack.

Key points:

- Persistance: keeps its Value even after the function exists
- Scope:
  - Inside a function: Local to that function but Persists.
  - Outside a function: Limited to the file (Internal Linage).
- Default Initialization: zero if not Explicity initialized.
- Memory Storage: Stored in the static memory (data Segment)

Scanned with OKEN Scanner

Pointer to a function:

```
void display()
{
  cout << "Hello";
}
int main()
{
declar. → void (*fp)();
initiali. → fp = display;
all → (*fp)();
}
```

Example:

```
int max(int x, int y)
{
  return x>y ? x : y;
}
int min(int x, int y)
{
  return x<y ? x : y;
}

int main()
{
  int (*fp)(int, int);
  fp = max;
  (*fp)(10, 5);
  fp = min;
  (*fp)(10, 5);
```

Introduction to OOPs — Object-Oriented Programming

~~Principles~~

Modular Programming — OOP

Bank
- Open_Acc()
- deposit()
- withdraw()
- checkbal()
- applyloan()

Govt
- Electric: newconc(), close(), Bill pay()
- Water
- Educ.
- Transport

Bank
- deposit()
- Withdraw()

Principles of Object-Orientation

1. Encapsulation (Data Hiding)

Encapsulation means hiding the internal details of the class and allowing access only through specific ~~functions~~ functions. It protect data from accidental changes

Real life Example

An ATM machines hides its ~~natural~~ internal process and only limited access through buttons

2. Abstraction

Abstraction means Showing only the important details and hiding ~~and~~ unnecessary Complexity.

Real life Example

When driving a car You only use the steering and Pedals. You don't See the engine's internal working

3. Inheritance (Reusing features)

Inheritance allows a child Class to inherit properties and behaviors from a parent class, reducing code repetition.

Real life Example

A child inherits traits like eye colour from their parents instead of developing them from Scratch.

4. Polymorphism (Many forms)

Polymorphism allows the same function behave diffrently based on the Context.

Real life Example

You are The Same Person, but

- As a Student, You Study
- As a Player, You Play Sports
- As a friend, You talk

Class and Object

A class is a blueprint for Creating objects. It defines data members (Variables) and members functions (methods) that operate on the data.

A class does not occupy memory until an object of it is Created.

Syntax:

```
class ClassName{
Public:
   // Data Member Variables
   int Data;
   // Member Functions (Methods)
   void display() {
     cout<<"Data:"<<Data<<endl;
   }
};
```

Object

An object is an instance of a class. When an object is created, it gets its own copy of the class's data members and can use its member functions.

Example

```
#include<iostream>
using namespace std;
class car{
Public:
   string brand;
   int speed;
   void showdetails() {
     cout<<"Brand:"<<brand<<" Speed:"<<
     speed<<"km/h"<<endl;
   }
};
int main() {
   car car1;

   car1.brand = "BMW";
   car1.speed = 200;
   car1.showdetails();
   return 0;
}
```

Key Points

1. Class = Blueprint / Template
2. Object = Instance of class
3. Class members (Variables & Functions) define behaviour and properties
4. Objects access members using the dot (.) operator

If no access specifier (public, private or protected) is mentioned, then class members are private by default.

#for Struct, members are public by default

Pointers to object

```
int main()
{
    Rectangle r;
    Rectangle *p;
    p = &r;
    r.length = 10;
    p-> length = 10;
```

```
class Rectangle
{
public:
    int length;
    int breadth;

    int area(){
    return length * breadth;
    }
    int Perimeter(){
    return 2 * (length * breadth)
    }
};
```

can use
p->[ ];
(*p).[ ];
and
p->[ ]();
(*p).[ ]();

Dynamic Allocation (Heap memory)

```
# Same class
int main(){
    Rectangle *p = new Rectangle;
        or
    Rectangle *p;
    p = new Rectangle;

    p-> length = 15;
    p-> breadth = 10;
    cout << p->area();

    delete p;
    p = nullptr;
    return 0;
}
```

Data Hiding

- Main Purpose: Prevent accidental or improper modifications of data.
- Focus: Ensures data integrity by Restricting direct access.
- Example: Making a Bank Balance Private Ensures it can only be modified through proper Methods, preventing unintended change

Accessor - getxxx
Mutator - setxxx

Scanned with OKEN Scanner

Constructor

A Constructor is a Special member function of a class that is automatically called when an object of that class is created. It is used to initialize object

Key features of a Constructor:

1. Same name as the class
2. No return type (not even void)
3. Called automatically when an object is created

Types of Constructor:

1. Default Constructor – Takes no arguments and initializes members with default values.

2. Parameterised Constructor – Take arguments to initialize an object with specific values

3. Copy Constructor – Creates a new object as a copy of an existing object.

4. Move Constructor – Creates a new object Moves Resources from a Temporary object instead of copying (introduced in C++11).

Example

```
#include<iostream>
using namespace std;
class Rectangle
{
public:
  int length, breadth;
  // 1. default Constructor
  Rectangle() {
    length = 0;
    breadth = 0;
  }
  // 2. Parameterized Constructor
  Rectangle(int l, int b) {
    length = l;
    breadth = b;
  }
  // 3. Copy Constructor
  Rectangle(Rectangle &rect) {
    length = rect.length;
    breadth = rect.breadth;
  }
  // 4. Move Constructor
  Rectangle(Rectangle &&rect){
    length = rect.length;
    breadth = rect.breadth;
    rect.length = 0;
    rect.breadth = 0;
  }
  int area() {
    return length * breadth;
  }
  void display() {
    cout<<"
    ";
  }
};
int main() {
  Rectangle r1;
  r1.display();
  Rectangle r2(10,5)
  r2.disp...();
  Rectangle r3 = r2;
  r3.display();
  Rectangle r4 = move(r2);
  r4.display();
  r2.display();
  return 0;
```

Constructor

A Constructor is a Special member function of a class that is automatically called when an object of that class is created. It is used to initialize object

Key features of a Constructor

1. Same name as the Class
2. No return type (not even void)
3. Called automatically when an object is created

Types of Constructor:

1. Default Constructor – Takes no arguments and initializes members with default values.

2. Parameterized Constructor – Take arguments to initialize an object with Specific Values

3. Copy Constructor – Creates a new object as a copy of an existing object

4. Move Constructor – Creates a new object Moves Resources from a Temporary object instead of Copying (introduced in C++11).

Example

```
#include<iostream>
using namespace std;
class Rectangle
{
public:
  int length, breadth;
  // 1. default Constructor
  Rectangle() {
    length = 0;
    breadth = 0;
  }
  // 2. Parameterized Constructor
  Rectangle(int l, int b) {
    length = l;
    breadth = b;
  }
  // 3. Copy Constructor
  Rectangle(Rectangle &rect) {
    length = rect.length;
    breadth = rect.breadth;
  }
  // 4. Move Constructor
  Rectangle(Rectangle &&rect) {
    length = rect.length;
    breadth = rect.breadth;
    rect.length = 0;
    rect.breadth = 0;
  }
  int area() {
    return length * breadth;
  }
  void display() {
    cout << "..." << ...
  }
};
int main() {
  Rectangle r1;
  r1.display();
  Rectangle r2(10,5);
  r2.disp...();
  Rectangle r3 = r2;
  r3.display();
  Rectangle r4 = move(r2);
  r4.display();
  r2.display();
  return 0;
}
```

Destructor (~)

A destructor is special member function of a class that is automatically called when an object is destroyed.
It is used to Release Resource like memory, file handles, or database connections that were allocated during the object's lifetime.

When Do we need a Destructor?

1. When using new to dynamic Memory Allocation
   - If we allocate memory using new, it does not get freed automatically when the function Ends. The Destructor Ensures delete is called to prevent memory leaks.

2. When Working with File Handling
   - If an object opens a file, the destructor Ensures the file is properly closed before the program exists.

3. When Managing Database Connections
4. When Handling Network Sockets or Threads.

All Types of function

```
class Rectangle
{
  Private:
    int length;
    int breadth;
  Public:
    Rectangle();                          } Constructor
    Rectangle(int l, int b);              }
    Rectangle(Rectangle & r);             }
    void setlength(int l);                } mutator
    void setbreadth(int b);               }
    int getlength();                      } Accessors
    int getbreadth();                     }
    int area();                           } Facilitators
    int Perimeter();                      }
    int isSquare();   ---> Enquiry
    ~Rectangle();     ---> Destructor
```

Scope Resolution Operator

The Scope Resolution operator (::) is used to access a global Variable, class member or a function that is outside of Current Scope.

Example 1) Global Variables

```
int x=10;
int main(){
  int x=20;
  cout<<x; // 20
  cout<<::x; //10
```

Scanned with OKEN Scanner

Example ② Defining Class function Outside The function

```
class Rectangle {
  Private:
    int length;
    int breadth;
  Public:
    int area(){
      return length * breadth;
    }
    int Perimeter();
};
  int Rectangle::Perimeter(){
    return 2*(length + breadth);
  }
```

inline function

An inline fundion is a function in C++ that the compiler attempts to Expand at the Point Where it is called, instead of Performing a regular function call. This helps Reduce call overhead and can improve Performance for Small functions

* function defined inside a class are implicity inline by default. ~~even~~ Compiler Treat them as inline function

Syntax

```
class Test{
  Public:
    void func1(){
      cout<<"Inline";
    }
    void func2();
};
void Test::func2(){
  cout<<"not-Inline";
}
```

This's Become now Inline ↓

```
inline void func2();
```

(If ✗)

Ex ②

```
#include<iostream>
using namespace std;

inline int square(int x){
    return x * x;
}
int main(){
cout<<"Square of 5:"<<square(5)<<endl;
return 0;
}
```

use ~~fnline~~ inline keyword b4 func. definition.

Best suited for Small and frequently called functions

This Pointer

This is a special pointer available inside class functions It stores the address of the object that is calling the functions

```
class Rectangle {
Private:
    int length;
    int breadth;
Public:
    Rectangle (int length, int breadth) {
    length
        this -> length = length;
        this -> breadth = breadth;
    }
```

Same for breadth

Because of this The length is

Difference B/w Structure and class

Key diff.

Feature	Struct	class
Default Specifier	Public (members are Public by default)	Private (members are Private by default
usage	mostly used for Simple data Structures	used for oops
Encapsulation	Limited (Everything is Public unless specified)	Strong Encapsulation (default is Private)
Inheritance	Public by default	Private by default

Operator Overloading

Operator Overloading allows You to Redifine How operator Works for user defined types (like classes and Structures). It helps make code more intutive by allowing object to use Operator just like built in data types.

Syntax

```
return_type operator op (parameters) {
    // implementation
}
```

- op is The operator being overloaded (+, -, *, /, ==, etc)
- The function can be member function or a friend function

Example

```
class Complex {
Private:
    int real;
    int img;
Public:
    Complex (int r=0, int i=0) {
    real=r;
    img=i;
    }
    Complex operator + (Complex& x) {
    Complex temp;
    temp.real = real + x.real;
    temp.img = img + x.img;
    return temp;
    }

int main() {
    complex c1(3,7)
    complex c2(5,4)
    Complex c3;
    c3 = c1.add(c2);
    c3 = c1 + c2;
```

Friend function

A friend function is a function that is not a member of a class but has access to its private and protected member. it is declared inside the class using the friend keyword

Syntax

```
class classroom {        // class name
    Private :
        int data;
    Public :
        friend return-type functionname (Parameters)
};
```

friend function is declared inside the class but defined outside
It does not have pointer since its not a member function
It can access private / protected member of the class.

Operator Overloading using Friend Function

Example

```
class complex {
  Private:
      int real;
      int img;
  Public:
friend Complex operator + (complex c1, complex c2)
};
```

```
Complex operator+ (complex c1, complex c2) {
  Complex t;
  t.real = c1.real + c2.real;
  t.img = c1.img + c2.img;
  return t;
}
```

Insertion operator Overloading

The insertion Operator (<<) Overloading is used with cout for output To Enable it to work with user defined classes, We Overload it as a friend function

Example

```
class complex {
  Private:
      int real;
      int img;
  Public:
     Friend ostream & Operator << (ostream & o, Complex & c1)
};
  ostream & Operator << (ostream & o, Complex & c1) {
      cout << c1.real << "+i" << c1.img; return out;
  }

# Can also use

Friend void operator << ——//——— ;

void Operator<< ——//——— {
    cout << ——//——;
}
```

Scanned with OKEN Scanner

Extraction Operator Overloading

The Extraction Operator (>>) is used with Cin to take input. To make it work with user defined classes, We overload it as a friend function

```
class Complex {
  private:
     int real;
     int img;
  public:
     friend istream & operator >> (istream&in, Complex c);
};

istream & operator >> (istream & in, Complex &c){
    cout<<"Enter real and imaginary number:";
    cin >> c.real >> c.img;
}
```

Inheritance

Inheritance is a fundamental Concept of Object Oriented Programming (OOP) that allows a class (child / derived class) to acquire the Properties and behaviour of another class (Parents / base class) It helps in Code Reusability and Extensibility

Key Concept

1. Base Class (Parent class): The Existing class from which Properties are inherited.
2. Derived Class (Child class): The new class that inherits from the Base class.
3. Access Specifiers (Public, Protected, Private): Controls How Base Class members are inherited.

Syntax:

```
class Base {
  // Base class Prop. and Methods
};

class Derived : public Base {
  // Derived class inherits Base class Properties
};
```

Example

```
class Base {
    Public:
      int x;
      void Show() {
          cout<< x;
      }
};
class Derived: public Base {
    Public:
      int y;
      void display(){
          Cout << x<<" "<<y;
          }
};

int main() {
  Base b;
  b.x=5;
  b.Show(); ⟶ 5
  Derived d;
  d.x = 10;
  d.y = 20;
  d.Show(); ⟶ 10
  d.display(); ⟶ 10 20
```

Constructors in inheritance

In inheritance, constructor play a crucial role in initializing the base class and derived class objects. Here's how Constructors behave in inheritance:

1. Execution Order of Constructor
   - When a derived class object is created the Base class Constructor is Executed first followed by the derived class Constructor
   - The base class constructor initializes the base class part of the derived object before the derived class Constructor executes

Types of Constructor invocation

(a) Default Constructor (No Arguments)

- If the Base class has a default Constructor, it is automatically called when the derived class object is Created

```
class Base{
  Public:
     Base() { cout<<" Base Constructor"; }
};
class derived: public Base{
 Public:
   derived() { cout <<"derived Constructor";}
};
 int main(){
   derived obj;
   return 0;
}
```

Output
Base Constructor
derived Constructor

Scanned with OKEN Scanner

(b) Parameterized Constructor

- If the Base class has a Parameterised Constructor, the derived class must explicity call it in its ~~Constructive~~ Constructor.

```
class Base {
 Public:
  Base (int x) { Cout << " Base Constructor" << x << endl; }
};
class derived : Public Base {
  Public:
   derived (int a) : Base (a) { // Calling Base constructor
          cout << "derived Constructor \n";
   }
};
 int main () {
   derived obj (10);
   return 0;
}
```

output
Base Constructor : 10
derived Constructor :

(c) Constructor Overloading

- The base and derived classes can have Overloaded Constructor

```
class Base {
 Public:
   Base () { cout << "default Base Constructor \n"; }
   Base (int x ) { Cout << " Param. Base Constructor: " << x << endl; }
class derived : public Base {
 Public
   derived () : Base (20) {
          Cout << " derived Constructor\n";
   }
};
```

```
int main () {
   derived obj;
   return 0;
}
```

output
Param Base Constructor: 20
derived Constructor

isA and hasA

In Object-Oriented Programming (OOP) isA and hasA relationships define ~~the~~ How classes are related to Each Other.

1. isA Relationship (inheritance)
   - Represents a generalization - specialization relationship
   - Used When one class is a type of Another class
   - Implemented using inheritance (Public, Protected, Private)

```
class Vehicle {

};
class car : Public Vehicle {

};
```

here "car isA Vehicle" → car inherits Properties of Vehicle

Scanned with OKEN Scanner

2. Has-A Relationship (Composition / Aggregation)

- Represents a "Has-A" relationship.
- One class has an instance of another class as a data member.
- Implemented using composition (strong ownership) or aggregation (weak ownership).

Example:

```
class Engine {
public:
    ...
};

class Car {
private:
    Engine engine;
};
```

Here: Car has-a Engine → Car contains an Engine object

Access Specifiers

Access specifiers define the visibility and accessibility of class members (variables and functions).

There are three main access specifiers:

Access Specifier	Accessible within class	Accessible in derived class	Accessible outside class
Public	✓ Yes	✓ Yes	✓ Yes
Protected	✓ Yes	✓ Yes	✗ No
Private	✓ Yes	✗ No	✗ No

1. Public Access Specifier
   - Members declared as public can be accessed from anywhere inside the class, in derived classes, and from outside the class.

```
class Demo {
public:
    int x;
    void show() { cout<<"x = "<< x <<endl; }
};
int main(){
    Demo obj;
    obj.x = 10;      ✓ Accessible
    obj.show();      ✓ Accessible
    return 0;
}
```

2. hasA Relationship (Composition / Aggregation)

- Represents a Whole Part relationship.
- Used When one class Contains onother class as a data member
- Implemented using Composition (strong dependency) or aggregation (loose dependency)

Example

```
class Engine {
public:
};
class car {
Private
  Engine engine;
};
```

here (car hasA Engine) — (car Contains an Engine object)

Access Specifiers

Access Specifier define the visibility and accessibility of class members (variables and functions)

C++ Provides Three main access specifier

Access Specifier	Accessible inside class	Accessible in derived class	Accessible Outside class
Public	✓ Yes	✓ Yes	✓ Yes
Protected	✓ Yes	✓ Yes	X No
Private	✓ Yes	X No	X No

1. Public Access Specifier
   - Member declared a public can be accessed from onywher (inside the class, in derived classes, and from outside the clas

```
class Demo {
public:
   int x;
   void show() { cout << "x = " << x << endl; }
};
int main() {
  Demo obj;
  obj.x = 10;        ✓ Accessible
  obj.show();        ✓ Accessible
  return 0;
}
```

Scanned with OKEN Scanner

2. Protected Access Specifier

Member declared as Protected can be accessed:

- ✓ inside the same class
- ✓ inside derived classes
- ✗ Not accessible from Outside the class:

Example

```
class Base {
Protected:
    int y;
Public:
    void Sety() (int val) { y = val; }
};
class Derived : Public Base {
  Public:
    void Show() { cout << "y = " << y << endl; }
};          // ✓ Accessible

int main() {
  Derived obj;
  obj.Sety(20);
  obj.Show(); // obj.y = 30
  return 0;
}
```

3. Private Access Specifier

- Member declared as Private can only be accessed inside same class

- They cannot be accessed in derived classes or from outside the class

```
class Base {
Private:
    int z;
Public:
    void Setz(int val) { z=val; } // ✓
    void Show() { cout<<"z = "<<z<<endl; };
int main() {
  Base obj;
  obj.setz(30);
  obj.show();
  //obj.z=10 ✗ not Accessible
  return 0;
}
```

Scanned with OKEN Scanner

Types of Inheritance

There are five main Type of inheritance in C++

1. Single Hexitance
   - A Single derived class inherits from a Single base class.

Example

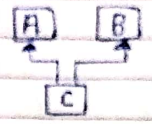
```
class A {
};
class B : public A {
};
```

2. Multilevel Inheritance
   - A class is derived from another derived class (like a chain)

Example

```
class A {
};
class B : Public A {
};
class C : Public B {
};
```

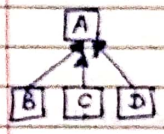
3. Multiple Inheritance
   - A Single derived class inherits from multiple base classes



Example:

```
class A {
};
class B {
};
class C : Public A, Public B {
};
```

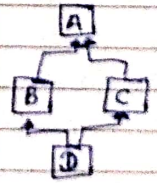
4. Hierarchical Inheritance
   - Multiple derived classes inherits from a Single Base class



Example:

```
class A {
};
class B : Public A {
};
class C : Public A {
};
class D : Public A {
};
```

5. Hybrid (Virtual) Inheritance
   - A Combination of Multiple and Multilevel inheritence



- Solves diamond Problem using Virtual Base class.

Scanned with OKEN Scanner

Virtual

Example # Diamond Prob. (without Base class)

```
class A { public: void Show() { cout << "Func of A" << endl; }
};
class B: Public A { };
class C: Public A { };
class D : Public B, Public C { };

int main() {
D. obj;
obj. Show() // X Error Ambigious (two copies of A::show)
```

Solution Virtual Base class

```
class A {
Public:
    void Show() { cout << " Func of A " << endl;
};
class B : Virtual public A { };
class C : Virtual public A { };
class D : public B, public C { };

int main(){
  D. obj;
  Obj. show(); // ✓ No ambiguity
```

output

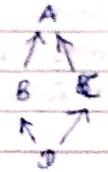
Func of A

Virtual Base Class in C++

A virtual Base class is used in multiple inheritance to prevent the duplication of the base class when it is inherited through multiple paths.

a Why is it Needed? (The Diamond Problem)

When a class is inherited by two different classes and a third class inherits both of them, it may create two copies of the Base class, leading to ambiguity.



- D gets two copies A, one from B and one from C
- This ~~Creat~~ Causes ambiguity when trying to access A's member

Solution declare A as a Virtual ~~class~~ base class to Ensure that only one Copy of A exists in D.

Syntax

```
class A{ };
class B : Virtual Public A {};
class C : Virtual Public A { };
class D : Public B, Public C { }; // Only one Copy of A exists
```

Prevent duplicate Copies
Resolve ambiguity
Ensure only one Shared instance
Uses the Virtual Keyword

Virtual

Example # Diamond Prob. (Without Base class)

```
class A { public: void show() { cout << "Func of A" << endl; }
};
class B : public A { };
class C : public A { };
class D : public B, public C { };

int main() {
D obj;
obj.show() // X Error Ambigious (Two Copies of A::show)
```

Solution Virtual Base class

```
class A {
public:
    void show() { cout << "Func of A" << endl;
};
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };

int main(){
D obj;
obj.show(); // ✓ No ambiguity
```

output

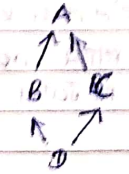
Func of A

Virtual Base Class in C++

A virtual base class is used in multiple inheritance to prevent the duplication of the base class when it is inherited through multiple paths.

Why is it Needed? (The Diamond Problem)

When a class is inherited by two different classes and a third class inherits both of them, it may create two copies of the base class, leading to ambiguity.



- D gets two copies A, one from B and one from C
- This causes ambiguity when trying to access A's member

Solution declare A as a Virtual base class to Ensure that only one copy of A exists in D.

Syntax

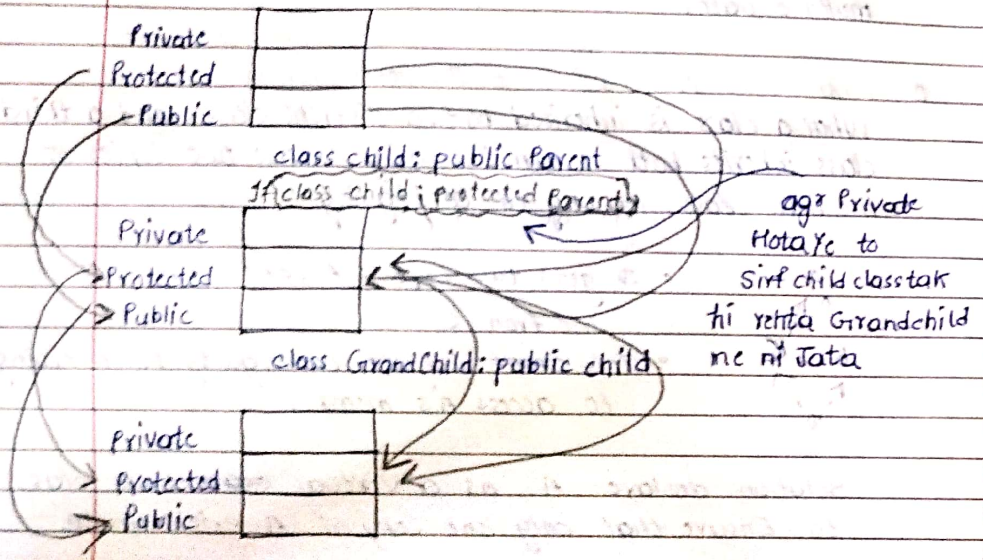
```
class A{ };
class B : virtual Public A{ };
class C : Virtual Public A{ };
class D : Public B, Public C { }; // Only one copy of A exists
```

Prevent duplicate copies
Resolve ambiguity
Ensure only one shared instance
Uses the virtual keyword

Scanned with OKEN Scanner

Ways of Inheritance

class Parent



Generalization and Specialization

Feature	Generalization	Specialization
Approach	Bottom-Up (Merging Common features)	Top/down (Adding new features)
Purpose	Combines Similar classes into one general class	Creates a Specialized Version of a general class
Example	car + Bike → Vehicle	Vehicle → car, Bike, Truck
Inheritance Type	Base class is created	Derived class is Created

Real Life Example

✓ Generalization

- A Smartphone and a laptop Both have a Battery, processor, and Screen
- Instead of duplicating these features, We make a general class "Electronic device" and let both inherit from it

✓ Specialization

- A Vehicle Can be a car, bike, or truck
- EOch has specific features (car has AC Truck has heavy load capacity)

Base class pointer derived class object

Syntax:

```
Derived b;
Base *ptr = &b;
```

Why is it Allowed?

- A derived object Contains all members of base
- A Base class pointer can store the address of a Derived class object (Upcasting)
- However, ptr can only Access Base class members, not derived class Specific ones.

Scanned with OKEN Scanner

Example:

```
#include<iostream>
using namespace std;

class Base {
  public:
    void Show() { cout<<"Base class"<<endl; }
};
class Derived: public Base {
  public:
    void display() { cout<<" derived class \n"; }
};

int main() {
  Derived b;
  Base *ptr = &b;

  ptr->show(); ✓ Allowed (Base class function)
  // ptr->display(); X not Allowed (Base Pointer can't Access derived member)
```

Why Ptr can't Access derived members?

- Ptr is of type Base*, so the Compiler only allow access to Base members
- Even though it Points to a derived object, it doesn't "see" the derived class members unless You use Virtual functions

What is about downcasting?

If You try,

```
Base a;
Derived *ptr = &a; // X not Allowed Compilation Error
```

This will no Work Because:

1. A Base class object does not have derived class members
2. A derived* expects an object with all derived features.

Example:

```
#include<iostream>
using namespace std;

class Base {
  public:
    void Show() { cout<<"derived-class"<<endl; }
};
class Derived : public Base {
  public:
    void display() { cout<<"derived class \n"; }
};

int main() {
  Derived b;
  Base *ptr = &b;

  ptr->show(); ✓ Allowed (Base class function)
  // ptr->display(); x not Allowed (Base Pointer
                     can't Access derived member)
```

Base

Why Ptr can't Access derived members?

- Ptr is of type Base*, so the Compiler only allow access to Base members
- Even though it Points to a derived object, it doesn't "see" the derived class members unless You use Virtual functions

What is about downcasting?

If You try

```
Base a;
Derived *Ptr = &a; // x not Allowed Compilation Error
```

This Will no Work. Because:

1. A Base ~~class~~ object does not have derived class members
2. A derived* expects an object with all derived features.

Scanned with OKEN Scanner

Why we go to Runtime Polymorphism
can't we go
Compile time polymo

Pointers Virtual
=function

Polymorphism

Function Overiding

Function Overiding is a feature in oops where a derived (child) class provides a specific implement ation of a function that is already defined in its base (Parent) class.

Key points:

1. Same Function Signature: The Overidden function in the child class must have the same name and parameters as in the Parent class.

2. Inheritance Required: function Overriding occurs only in an inheritance Relationship

3. Virtual functions: The Parent class function should be marked as virtual to allow Overriding.

4. Polymorphism: function Overriding is an Example of runtime Polymorphism

Example

```
class Parent {
public:
    void show() {
        cout << "inside Base";
    }
};
class child : public Parent {
public:
    void show() {
        cout << " inside child";
    }
};
int main() {
derived d;
d.show(); → ~~cout~~ inside child class
return 0;
}
```

If instead of This

```
int main() {
child d;
Parent *ptr = &d;
d.show(); → inside child
ptr -> show(); → inside Parent
return 0;
}
```

If we have used virtual keyword ~~infront~~ infront of void inside (base) Parent class Then it will Return

inside child

Virtual Functions

In c++, we use virtual keyword to enable runtime polymorphism ensuring that the overriden function in the derived class is called instead of the base class function when using a Base class pointer or reference.

Why use Virtual?

By default, c++ uses early binding (compile time binding) meaning function calls are resolved based on the type of pointer / reference at compile time.

- When a function is declared a virtual in the base class, late binding (runtime binding) occurs, allowing the derived class function to be called instead

Eg

Same as previous code only change is

```
class Parent {
public:
    virtual void Show() {
        cout << "inside Base";
    }
};
int main
Child d;
parent *p = &d;
p -> Show(); --> inside child
```

Polymorphism is one of the core Concept of oop that allows object of different classes to be treated as objects of a common Superclass. It enables a single interference to Represent data types or structures, making code ~~same~~ more flexible and reusable.

Two main Type of Polymorphism

1. Compile-time polymorphism (Static Binding)
   - The function to be determined at compile time.
   - Achieving using:
     - Function Overloading
     - Operator Overloading
     - Templates (in Some Cases)

Example of function overloading

```
class Math {
public:
    int add(int a, int b) { return a+b; }
    double add(double a, double b) { return a+b; }
};
```

2. Runtime Polymorphism (Dynamic Binding)
   - Achieving using method overiding and Virtual functions
   - The function call is resolved at runtime

Example

```
class Parent {
    public:
        virtual void show() { cout<<"parent class"<<endl;}
};
class child: public Parent {
  public:
      void show() override { cout<<"child class"<<endl;}
};
```

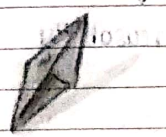
- Not mandatory, but using is best Practice
- Helps avoid subtle bugs and improves maintainability.

```
class Shape {
public:
    virtual void draw()=0; // Pure virtual function
};

class circle : public Shape{
public:
    void draw() override {
        cout << "Drawing circle" << endl;
    }
};
class Rectangle : public Shape{
public:
    void draw() override {
        cout<<"Drawing Rectangle"<< endl;
    }
};
```

```
int main() {

Shape * *p = new circle();
p -> draw(); ——— Drawing circle
p = new Rectangle();
p -> draw(); ——— Drawing Rectangle
```



Abstract classes

An Abstract class is a class that contains at least one pure Virtual function. It cannot be instantiated, meaning, you cannot create objects of an abstract class directly instead, it serves as a blueprint for derived classes.

Syntax of an Abstract class

```
class Abstractclass {
  public:
     virtual void purefunction() = 0; // Pure Virtual function
};
```

- The =0 makes purefunction() a pure virtual function.
- Any class with at least one pure Virtual function is Abstract.

Scanned with OKEN Scanner

interface (Java)

Base	Base	Base
All concrete	Some concrete	All pure
	Some pure Virtual	Virtual
↓	↓	↓
reusability	reusability	polymorphism
	polymorphism	

Friend Function and Friend Class

1. Friend Function

A Friend Function is a function that is not a member of a class but has access to its private and protected members. It is declared inside the class with the keyword friend but defined outside the class.

Example

```
#include<iostream>
using namespace std;

class A {
  private:
    int num;
  Public:
    A(int x): num(x) { }        // Member initialization

friend void show(A obj);

};
  void show(A obj) {
     cout<<"The Private number is"<< obj.num<<endl;
};
```

```
int main() {
  A obj(10);
  show(obj);
  return 0;
}
```

- It is declared inside the class but defined outside
- It is not a member of the class
- It can Access private and protected members of the class.

2. friend class

A friend class is a class that can Access the private and Protected members of another class

Example

```
class B;
class A {
private:
    int num;
public:
    A(int x): num(x) { }

  friend class B;
};
class B {
public:
   void show(A obj) {
      cout<<"The private member is:"<<obj.num
          <<endl;
  } };
```

```
int main() {
  A obj(20);
  B b;
  b.show(obj); // Accessing
  return 0;    // Private
}              // members
```

Scanned with OKEN Scanner

- A friend class can Access private and Protected members of another class
- It is useful when two classes need to share data
- Friendship is not inherited, meaning derived classes of a friend class do not get access.

Static Members

Static members in C++ belongs to the class Rather than any Specific object. They Retain their values across all instances of the class.

1. Static members

- declared using the static keyword inside the class.
- Shared among all objects of the class (i.e. only one copy exists)
- Must be defined outside the class.

Example

```
class A {
public:
  static int count;

  A(){
    count++;
  }
};
int A :: count = 0;

int main(){
  A obj1, obj2, obj3;
  cout << "Total objects created:" << A::count << endl;
  return 0;
}
```

Output
Total object created : 3

Key points
- Shared by all objects of the class.
- Must be defined outside the class
- Can be accessing using
  `ClassName::memberName.`

2. Static Member function

- Declared using the static keyword inside the class
- Can only access static data members and other static functions
- Called using
  `className::FunctionName.`

```
class A {
private:
  static int count;
public:
  static void show(){
    cout << "Count:" << count << endl;
  }
};
```

Scanned with OKEN Scanner

```
int A::count = 5;

int main(){
  A::showCount(); // calling static function without object
  return 0;
}
```

Output

count: 5

Key points

- Can be called Without Creating An Object

  `ClassName::FunctionName()`

- Cannot access non-static members of the class

`A(int x): num(x) {}` → Constructor with member initialization list

This syntax Represents a Constructor With a member initialization list in C++. It is used to initialize class data members before entering the constructor body

Breakdown:

`A(int x): num(x) { }`

- `A(int x)` → Constructor that takes an Integer parameter
- `: num(x)` → member initialization list, which initialize num with x before entering the Constructor body
- `{}` → Empty Constructor body (since initialization is already done).

Inner Nested class

An inner Nested class is a class defined within another class It is also called nested class or inner class

Syntax

```
class Outer {
public:
    class Inner {
    public:
        void display(){
            cout<<"inside inner class"<<endl;
        }
    };
    void show(){
        cout<<" inside Outer class"<<endl;
    }
};
```

How to use it

```
int main(){
    Outer::Inner obj  //Create object of Inner class
    obj.display();
    Outer Out;
    Out.show();
    return 0;
}
```

Scanned with OKEN Scanner

Important Points

1. Nested class can Access only public and Static member of the Outer class directly.
2. Outer class cannot Access private members of the inner class directly.
3. Nested class is independent of the Outer class object.

Exception Handling

Exception Handling in C++ is done using try, catch and throw. It helps in Handling Runtime Errors to prevent program crashes.

Exception Handling is a Mechanism to Handle runtime errors (like division by zero, invalid input etc) so that the Program does not Crash unexpectedly.

Why do We Need Exception Handling ?

Without exception Handling a program may terminate unexpectedly when an error occurs Example:

```
#include<iostream> using namespace std;
int main(){
    int a=10, b=0;
    cout<< a/b; // Error: division by zero
    return 0;
}
```

How Exception Handling Works

C++ Provides Three keywords for Handling exceptions

1. try → Defines a block of code that may cause an Exception
2. throw → Used to Raise (throw) an Exception
3. Catch → Defines a block that Handles this Exception.

Example

```
int a=10, b=0, ~~c=0~~;
try{
    if(b==0)
        throw 1
    cout<<" a/b;
}
catch(int e){
    cout<<"division by zero"<<endl;
}
```

Example 2nd

```
Int division(int a, int b){
    if(b==0)
        throw 1
    return a/b;
}
```

```
try{
    z=division(4,0)
    cout<<"z;
}
catch(int e){
    cout<<"div. by zero"
    <<endl;
}
```

Scanned with OKEN Scanner

All About Throw()

The `throw` keyword is used to signal (or raise) an exception when an error or unexpected condition occurs during program execution.

Syntax:

```
throw expression;
```

- expression can be an object or value representing the error.
- When throw is executed, the control is transferred to the nearest matching catch block.

What can be thrown?

- int, char, etc
- Standard library exception like runtime_error, invalid_argument
- user defined exception classes

Example using runtime_error

```
#include<iostream>
#include<stdexcept>
using namespace std;

int main(){
    try{
        throw runtime_error("Standard runtime error occured");
    }
    catch(const exception& e){
        cout<<"Caugh standard exception: "<<e.what()
        <<endl;
    }
    return 0;
}
```

Output

Caught Standard exception: Standard Runtime error occured

Example: Custom Exception class

```
#include<iostream>
#include<exception>
using namespace std;
class MyCustomException: public exception{
public:
    const char* what() const noexcept override{
        return "My custom exception occured";
    }
};
```

Scanned with OKEN Scanner

```
int main(){
    try {
        throw MyCustomException();
    }
    catch(const exception& e){
        cout << "Caught Custom exception: " << e.what()
             << endl;
    }
    return 0;
}
```

Output

Caught custom exception: My custom exception occurred!

Re-throwing Exception

What is it?
Rethrowing is when you catch an exception but decides to pass it up the call stack to another catch block, often higher in the call hierarchy.

Example: Re-throwing

```
#include<iostream>
using namespace std;

void testFunction(){
    try {
        throw 42;
    }
    catch(int e) {
        cout << "Caught in inner try: " << e << endl;
        throw; } }
```

```
try {
    testfunction();
}
catch(int e){
    cout << " Caught in Outer try: " << e << endl;
}
return 0;
}
```

Output:
Caught in inner try: 42
Caugh in Outer try: 42

Standard Exceptions (Built in)

1) Logic Errors (<stdexcept>)
   - invalid_argument
   - domain_error
   - length_error
   - out_of_range
2) Runtime Error
   - runtime_error
   - overflow_error
   - underflow_error
   - range_error
3) Other
   - bad_alloc
   - bad_cast
   - ios_base::failure

Scanned with OKEN Scanner

All about ~~Throw~~ Catch

A block that Handles exceptions thrown by a `throw` inside a `try` block.

Syntax

```
catch (Type Var) {
    // code to handle the exception
}
```

Multiple Catch blocks

You can have more than one Catch block to Handle different types of Exceptions after a Single try

Catch all handler

Written as `catch(...)`, this block catches any type of Exception if no Specific Catch matches

Order

C++ checks catch from top to bottom, so Specific catch block must come before General one like `catch(...)`

Reference in Catch

You can use a reference to avoid Copying the Exception

```
catch (const std::exception& e)
```

Catch With no Parameter

If the Parameter name isn't needed, you can omit it:

```
catch(int) {...}
```

Template function

Template functions in C++ allow you to write Generic and Reusable functions that can Operate with diffrent data types.
Instead of Writing multiple Overloads for Each type, you can define one function Template that works ~~again~~ for any type.

Syntax

```
template <typename T>
T max (T a, T b) {
    return (a>b)? a : b;
}
```

You can use class instead of typename — they mean the Same in this Context.

multiple template data type

```
template <class T, class R>
void add (T x, R y) {
    cout << x+y;
}
```

add (9, 11.63)
→ 18.63

Scanned with OKEN Scanner

What is a template class?

A Template class allows you to create a class that can work with any data type - int, float, String, etc without rewriting the code.

Syntax

```
template <class T>
class Class_name {
    T data;
public
    void Setdata(T val){
        data = val;
    }
    T getdata() {
        return data;
    }
};
```

Reusability → Write once use with anytype
Type Safety → No need for void* or unsafe casting
Clean code → Avoid duplicate classes for each type

Constant Qualifier

The `Const` Qualifier is used to define Constants - values that cannot be changed after initialization. It can be applied in many Context (variables, pointers, function parameters, member function etc) with different meanings

1. `Const` variable

```
const int x = 10;
```

- x is read-only variable; its value cannot be modified.
- must be initialized when declared

2. `Const` with pointers

a) pointer to const data

```
const int* ptr = &x;
//OR
int const* ptr = &x;
```

- You can change ptr to point somewhere else.
- But you cannot change the value it points to (*ptr = 20; is invalid)

b) Const pointer

~~int* pointer~~

```
int* const ptr = &x;
```

- You can change the value at that location (*ptr = 20; is allowed)
- But you cannot point pointer to another variable

What is a template class?

A Template class allows you to create a class that can work with any data type - int, float, string, etc without rewriting the code.

Syntax

```
template <class T>
class Class_name {
    T data;
public
    void setdata(T val){
        data = val;
    }
    T getdata() {
        return data;
    }
};
```

Reusability → Write once use with anytype
Type Safety → No need for void* or unsafe casting
Clean code → Avoid duplicate classes for each type

Constant Qualifier

The `Const` Qualifier is used to define constants - values that cannot be changed after initialization. It can be applied in many contexts (variables, pointers, function parameters, member function etc) with different meanings

1. `Const` variables

```
const int x = 10;
```

- x is read-only variable; its value cannot be modified.
- must be initialized when declared.

2. `Const` with pointers

a) pointer to const data

```
const int* ptr = &x;
//OR
int const* ptr = &x;
```

- You can change ptr to point somewhere else.
- But you cannot change the value if points to (*ptr = 20; is invalid)

b) Const pointer

~~int* pointer~~

```
int* const ptr = &x;
```

- You can change the value at that location (*ptr = 20; is allowed)
- But you cannot point pointer to another variable

c) const pointer to const data

```
const int * const ptr = &x;
```

- Niether the pointer nor the value it points to can be changed.

3. const function parameters

a) Pass-by-value

```
void func(const int x){
    // x cannot be modified
}
```

b) Pass-by-reference

```
void func(const int &x){
    //more efficient for large obj. Cannot modify x
}
```

4. Const in function declaration

a) Member function

```
class A {
    int x;
public:
    int getX() const{
        return x;
    }
};
```

The const at the end means the function will not modify any member variables (unless marked mutable).

5. const With Objects

```
const A obj;
obj.getX(); // only 'const' function can be called
```

- obj is read only, so only const member functions can be called on it.

6. constexpr (related to const)

- introduced in C++
- Value must be ~~declared~~ known at Compile time.

```
constexpr int size = 5;
```

diff in const & constexpr

const	constexpr

int x
cin >> x;
const int a = x;
OR
const int a = 10;
	constexpr int a = 10;
	(Value must be known before, at Compile time)

- Both possible

(Value might be known later, at runtime)

Scanned with OKEN Scanner

Streams

File Handling

File Handling in C++ is done using the fstream library which provide Three main classes

1. Open for Reading (ifstream)

```
#include<fstream>
using namespace std;

int main(){
    ifstream fin;
    fin.open("file.txt");
    if(!fin){
        cout<<"Failed to open file\n";
    }
    fin.close();
    return 0;
}
```

2. Open for Writing (ofstream)

```
#include<fstream>
using namespace std;
int main(){
    ofstream fout;
    fout.open("file.txt");
    fout<<"Hello World";
    fout.close();
    return 0;
```

3. Open for Both Reading and Writing (fstream

```
fstream file;
file.open("file.txt", ios::in | ios::out);
```

file opening Modes (ios::)

Mode	description
in	input (read)
out	Output (write)
app	Append to end
trunc	Delete old content
ate	Move to end after opening
binary	open file in binary mode

can combine mode using |, like ios::in | ios::out.

Scanned with OKEN Scanner

M.txt
Mehul Suthar
20

How to Read from file

```
#include<iostream>
#include<string>
#include<fstream>

int main(){
    fstream file("M.txt", ios::in | ios::out);
    int x;
    string str1;
    getline(File, str1);
    File >> x;
    Cout<< str1 <<endl<< x;
    File.close();
    return 0;
}
```

Output
Mehul Suthar
20

Serialization

Serialization is the process of Converting an object (like a class instance) into a format that can be easily stored (e.g., in a file or database) or transmitted (e.g., over a network). it's essentially "flattening" an object so it can be Saved or Shared and later "deserialised" back to its Orignal form.

Why Serialization?

- Saving object State: You can save the State of an object and Reload it later.
- Transmitting objects: When you need to Send Objects over a network, for example in client-Server Communication.

Types of Serialization:

1. Binary Serialization: Objects is Converted into a binary format (more Compact, faster)
2. Textual (or XML/JSON) Serialization: Object is Converted into a text-based format (Human-readable Slower).

Scanned with OKEN Scanner